Contents

Airbnb NYC — Module 3 Lab Guide

Scenario: New York short-term rentals. Predict the right price per night. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Airbnb New York**. Airbnb is a website where people rent rooms or whole apartments to tourists. A host puts a listing online. A guest books it for one or more nights.

In New York City: - **About 40,000 listings** are active right now. - **About 500,000 reviews** were written by past guests. - Every host picks their own price per night.

The problem

Hosts do not know the right price.

- Some hosts **over-price.** Their listing is too expensive. Nobody books it. The room stays empty. The host loses money.
- Some hosts **under-price.** Their listing is cheap. They get bookings, but they earn less than they could.

Both groups complain to Airbnb support every day. Support cannot answer all the calls. The Head of Host Operations is worried.

Your manager's request

Your manager, **Marcus** (Head of Host Operations, Airbnb NYC), tells you in a Monday meeting:

> "I do not want to set the price for the host. The host is free.
>
> But I want to **help** the host. When a new host posts a listing, I want to show one number on their screen: **'similar listings in your area charge $X per night.'**
>
> The host can pick a higher price, or a lower price. That is their choice. But they will know what the market looks like.
>
> If we give 10,000 new hosts a good price hint this year, we will save them from empty calendars and angry support calls."

Your team's job for the next 2 weeks (Module 3)

Marcus cannot do this alone. The data is **two big files** plus some quirks. The price column is dirty. Some addresses are missing. The bathrooms column has weird text.

Your job in Module 3: > **Turn the raw Inside Airbnb files into ONE clean file. The clean file will be used to train the price model in Module 4.**

The clean file is called `airbnb_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6). It will have about **38,000 rows**.

After Module 3

Module	What happens
Module 4	Train the price prediction model. Marcus finally gets his "fair price hint".
Module 5	Find groups of listings (cheap rooms, luxury whole apartments, etc.).
Module 7	Read the guest review comments. Find common praise and complaints.

You use the **same Airbnb NYC dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see two kinds of content:

1. WHAT / WHY / EXPECTED — explained in words

For every step you read: - **WHAT** you have to do (in normal language) - **WHY** you do it (the reason) - **HINTS** — the function names, the arguments to remember - **EXPECTED** — the result you should see if you did it right

2. Sometimes a tiny code skeleton with blanks

For the harder steps we give you a code skeleton with ___ blanks. **You fill in the blanks.**

Why no full code?

Because you will NOT learn if you copy-paste. - In Module 4, the model only works if YOU understand each step. - In your future job, nobody will give you the code. - Writing the code yourself takes 10 minutes more today, but saves 10 hours later.

Rule: If you finish a step in 30 seconds by copy-pasting from a teammate's notebook, do it AGAIN by yourself. Different variable names, same logic.

Visualizations — Two Modes

In every class you will make charts. There are **two reasons** to make a chart:

1. Exploratory charts (for YOU)

Made BEFORE you clean the data. To understand what the data looks like. - Fast, ugly, no labels needed. - Examples: `df.hist()`, `df['column'].value_counts().plot.bar()`.

> Goal: **"What does this data look like?"**

2. Explanatory charts (for MARCUS)

Made AFTER you understand. To EXPLAIN something to your manager. - Clean, labeled, one clear message. - Must have: title, x-label, y-label, source, and a 1-sentence takeaway.

> Goal: **"Marcus, look at this. This is the pattern."**

In every class you make BOTH kinds.

Your plotting toolkit (you learned this in Module 2 Class 5)

Plot	When to use it	Function name
Histogram	See the SHAPE of a numeric column	`plt.hist()` or `sns.histplot()`
Bar chart	Count of each category	`df['col'].value_counts().plot.bar()`
Box plot	See outliers in a numeric column	`sns.boxplot()`
Scatter plot	See the relationship between 2 numbers	`plt.scatter()` or `sns.scatterplot()`
Heatmap	See correlation between many columns	`sns.heatmap(df.corr())`
Map scatter	See points on a real map	`plt.scatter(lon, lat, c=price)`

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

You will work in **Google Colab**. That means: - You do NOT install Python. - You do NOT install pandas, scikit-learn, or anything. - You just open a notebook in your web browser.

Step 1 — Open a new Colab notebook

1. Go to https://colab.research.google.com
2. Sign in with your Google account.
3. Click **"New notebook"** (top left).
4. Name it `airbnb_module_3.ipynb`.

Step 2 — Connect to Google Drive

Colab files **disappear** when you close the browser. So you must save your work to **Google Drive**.

In the first cell:

```
from google.colab import drive
drive.mount('/content/drive')
```

A pop-up asks for permission. Click "Allow". After this, Drive is available at `/content/drive/MyDrive/`.

Step 3 — Create a project folder

In a new cell:

```
import os
os.makedirs('/content/drive/MyDrive/airbnb_lab', exist_ok=True)
%cd /content/drive/MyDrive/airbnb_lab
```

Your team will save ALL files in this folder.

Step 4 — Get the data

The data comes from **Inside Airbnb**. It is an independent project. The license is CC0 (public domain). You do NOT need an account.

Option A — Direct download in Colab (recommended):

1. Go to http://insideairbnb.com/get-the-data/ on your laptop.
2. Find the **New York City, New York, United States** section. Pick the latest scrape date.
3. Copy the URLs for two files:
   - `listings.csv.gz` (the detailed one, around 40,000 listings, 75 columns)
   - `reviews.csv.gz` (the detailed one, around 500,000 reviews)
4. In Colab:

```
import urllib.request
LISTINGS_URL = 'PASTE_URL_HERE'
REVIEWS_URL  = 'PASTE_URL_HERE'
urllib.request.urlretrieve(LISTINGS_URL, 'data/listings.csv.gz')
urllib.request.urlretrieve(REVIEWS_URL,  'data/reviews.csv.gz')
```

(Make a `data/` folder first with `os.makedirs('data', exist_ok=True)`.)

Option B — Upload by hand:

1. Download the two files on your laptop.
2. In Colab's file panel (left sidebar), upload them into a `data/` folder.

Step 5 — Test it

```
import pandas as pd
listings = pd.read_csv('data/listings.csv.gz')
print(listings.shape)
```

You should see something like `(40123, 75)`. The exact number changes by month, but it is around 40,000 rows and around 75 columns. You are ready.

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate `.parquet` files to Drive	Files in `/content/` disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.
Restart runtime if memory full	Runtime > Restart runtime.

Class 1 — Data Cleaning

> **Scenario reminder:** Marcus drops the raw Inside Airbnb files on your desk. The price column has `$` and `,` characters. The bathrooms column has weird strings like "1.5 baths" and "Half-bath". Your job today: clean the most important columns.

Your goal

Make the raw `listings.csv.gz` file USABLE. Fix the price column. Fix the bathrooms column. Find missing values. Decide what to keep.

Inputs

- `data/listings.csv.gz`
- `data/reviews.csv.gz`

Outputs

- `listings_step1.parquet` saved in your Drive folder
- 3+ exploratory charts in your notebook
- 1 explanatory chart for Marcus
- Notes in markdown about every decision you made

Phase A — Explore the data first (15 minutes)

Before you clean anything, **LOOK** at the data.

Exploratory chart 1 — How many listings per `room_type`?

- **Question:** "Of about 40,000 listings, how many are Entire home, Private room, Shared room, Hotel room?"
- **HINTS:**
  - Use `listings['room_type'].value_counts()`.
  - Then `.plot.bar()` on the result.
- **What you learn:** Most listings are "Entire home" or "Private room". The other types are rare.

Exploratory chart 2 — How many listings per `neighbourhood_group_cleansed`?

- **Question:** "How many listings in each borough (Manhattan, Brooklyn, Queens, Bronx, Staten Island)?"
- **HINTS:**
  - Use `value_counts()` then bar chart.
- **What you learn:** Manhattan and Brooklyn are the biggest. Staten Island is tiny.

Exploratory chart 3 — Missing values per column

- **Question:** "Which columns have the most missing data?"
- **HINTS:**
  - Use `listings.isna().sum().sort_values(ascending=False).head(20)`.
  - Plot it as a bar chart.
- **What you learn:** `bedrooms` is missing in about 30% of rows. Some date columns are also missing.

Exploratory chart 4 — A first look at `price`

- **Question:** "What does the raw price column look like?"
- **HINTS:**
  - Print `listings['price'].head(20)`.
  - You will see strings like `"$150.00"`, `"$1,200.00"`.
- **What you learn:** Price is NOT a number yet. It is text with a dollar sign and commas. You must clean it before you can plot a histogram.

Phase B — Clean the listings table (45 minutes)

Step 1 — Load both files

- **WHAT:** Load the listings file and the reviews file into two DataFrames.
- **HINTS:**
  - `listings = pd.read_csv('data/listings.csv.gz')`.
  - `reviews  = pd.read_csv('data/reviews.csv.gz')`.
  - Pandas reads `.csv.gz` directly. No need to unzip.
- **EXPECTED:** Two DataFrames. Listings around 40,000 rows. Reviews around 500,000 rows.

Step 2 — Look at the DataFrame

- **WHAT:** Check `.shape`, `.info()`, and `.head()` on listings.
- **HINTS:**
  - 75 columns is a lot. Use `listings.columns.tolist()` to see all names.
  - Look at the `Dtype` column. Many "number" columns are actually `object` (text).
- **EXPECTED:** You see that `price`, `bathrooms_text`, `host_since`, and `first_review` are all text.

Step 3 — Clean the `price` column

This is the most important column in the whole project. It must be a float.

- **WHAT:** Remove the `$` and the `,`. Then convert to float.
- **HINTS:**
  - `listings['price'].str.replace('$', '', regex=False)`.
  - Chain another `.str.replace(',', '', regex=False)`.
  - Then `.astype(float)`.
  - One line: `listings['price'] = listings['price'].str.replace('$','',regex=False).str.replace('`
- **WARNING:** If a cell is missing, the chain may crash. Wrap with `errors='coerce'` style. Or drop missing prices first.

- **EXPECTED:** `listings['price'].dtype` is `float64`. The minimum is around $10. The maximum can be $10,000+ (outliers).

Step 4 — Parse the `bathrooms_text` column

The column has weird strings. Examples: - `"1 bath"` - `"1.5 baths"` - `"Half-bath"` - `"Shared half-bath"` - NaN (missing)

- **WHAT:** Extract the number. Save as `bathrooms` (float).
- **HINTS:**
  - First, lowercase the column: `listings['bathrooms_text'].str.lower()`.
  - For "half-bath" rows, the number is 0.5. Use `.str.contains('half')` and set those to 0.5.
  - For the rest, extract the first number using regex: `.str.extract(r'(\d+\.?\d*)')`.
  - Convert to float.
  - Skeleton:

```
txt = listings['bathrooms_text'].str.lower()
is_half = txt.str.contains('half', na=False)
nums = txt.str.extract(r'(\d+\.?\d*)')[0].astype(float)
listings['bathrooms'] = nums
listings.loc[is_half, 'bathrooms'] = ___
```

  - Fill in the blank.
- **EXPECTED:** A new float column `bathrooms`. Min around 0, max around 10. About 1% still missing.

Step 5 — Fix the date columns

- **WHAT:** Convert `host_since`, `first_review`, `last_review`, `last_scraped` to real datetime.
- **HINTS:**
  - The function is `pd.to_datetime()`.
  - Add the argument `errors='coerce'`. If a cell is bad, it becomes NaT (Not a Time = missing). The code does not crash.
  - Use a `for` loop over the 4 column names. Do NOT write 4 separate lines.
- **WHY:** If dates are strings, you cannot subtract them. "How long has the host been on Airbnb?" is impossible.
- **EXPECTED:** After your code, `listings.dtypes` shows `datetime64[ns]` for those 4 columns.

Step 6 — Find missing values

- **WHAT:** Count missing values per column. Look at the top 15.
- **HINTS:** `.isna().sum().sort_values(ascending=False).head(15)`.
- **EXPECTED:** Something like:

```
bedrooms                12000
reviews_per_month        9000
first_review             9000
last_review              9000
bathrooms                 400
```

Step 7 — Drop rows with missing or zero price

- **WHAT:** A listing with no price (or price = 0) is useless for our model.
- **HINTS:**
  - First drop rows where `price` is NaN.
  - Then drop rows where `price` <= 0.
- **EXPECTED:** About 39,000 rows left.

Step 8 — Drop extreme price outliers

- **WHAT:** A few listings cost $10,000+ per night. They are not real.
- **HINTS:**
  - Look at the 99th percentile: `listings['price'].quantile(0.99)`.
  - Drop rows above some threshold (try $1,000 or $1,500).
  - Also drop rows below $10 (probably typos).
- **WHY:** Outliers will pull the average and ruin the model.
- **EXPECTED:** About 38,000 rows left.

Step 9 — Write down what you did

In a markdown cell, write: - Starting rows: ~40,000 - After dropping missing price: ~39,000 - After dropping price outliers: ~38,000 - WHY you removed each group.

Step 10 — Save to Drive

- **WHAT:** Save the cleaned `listings` DataFrame as parquet, inside your Drive folder.
- **HINTS:** `listings.to_parquet('/content/drive/MyDrive/airbnb_lab/listings_step1.parquet')`.

Phase C — Make ONE chart for Marcus (15 minutes)

He does not have time to read your code. He wants ONE picture.

Marcus's chart — "How does price look across the city?"

Make a histogram of `price` AFTER cleaning. Use a sensible upper limit (e.g. cut at $500 for the chart).

- **HINTS:**
  - `plt.hist(listings['price'], bins=50, range=(0, 500))`.
  - Add the title: `"NYC Airbnb price distribution — most listings between $60 and $250"`.
  - X-label: Price per night (USD).
  - Y-label: Number of listings.
  - Add a vertical line at the median: `plt.axvline(listings['price'].median(), color='red')`.
- **Takeaway for Marcus:** "Half of all listings charge less than $X per night. The market is very wide. A new host needs a hint based on their own neighborhood and room type, not the city average."

Common mistakes in Class 1

Mistake	What goes wrong
Forget to remove the `,` in `"$1,200.00"`	`astype(float)` crashes.
Forget `errors='coerce'` on `to_datetime`	Code crashes on one bad date.
Drop ALL missing rows in one go	You lose 80% of the data. Drop per column with care.
Cap price at $100	You delete real luxury listings. Use $1,000 or $1,500.
Save to `/content/` instead of Drive	File disappears when Colab disconnects.

Self-check before Class 2

- ☐ Both files loaded.
- ☐ `price` is float. Range looks sensible (10 to ~1500).
- ☐ `bathrooms` is float. "Half-bath" handled.
- ☐ The 4 date columns have dtype `datetime64`.

- ☐ At least 3 exploratory charts in your notebook.
- ☐ 1 polished chart for Marcus.
- ☐ You saved `listings_step1.parquet` to your Drive folder.
- ☐ You wrote down (in markdown) what you did and WHY.

Class 2 — Encoding and Scaling

> **Scenario reminder:** Marcus looks at your cleaned data. He is happy that price is now a number. But he says: "The model is a math model. It does not understand the word 'Entire home/apt'. Turn the words into numbers. Also, the price has a very long tail — fix that."

Your goal

Turn TEXT columns into numbers. Apply log-transform to price. Make all numeric columns about the same size.

Inputs

- `listings_step1.parquet` from Class 1

Outputs

- `listings_step2.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Marcus

Phase A — Explore the data first

Exploratory chart 1 — Distribution of `price` (raw)

- **Question:** "Most listings are $100-$300. But a few are $1,000+. Is the distribution skewed?"
- **HINTS:**
  - Use a histogram (`plt.hist()`).
  - Use `bins=50`.
- **What you learn:** The price column has a long right tail. This is why we will use log-transform.

Exploratory chart 2 — Distribution of `log_price` (after `np.log1p`)

- **Question:** "Does log-transform make the shape more normal?"
- **HINTS:**
  - Make a NEW column: `log_price = np.log1p(listings['price'])`.
  - Histogram it.
  - Compare to chart 1. The new shape should look like a bell.
- **What you learn:** Log makes the long tail manageable for the model.

Exploratory chart 3 — `room_type` counts

- **Question:** "Which room types are popular in NYC?"
- **HINTS:** `value_counts().plot.bar()`.
- **What you learn:** Most are "Entire home/apt" or "Private room". Shared rooms and hotel rooms are rare. (Maybe merge the rare ones?)

Exploratory chart 4 — `neighbourhood_cleansed` counts

- **Question:** "How many distinct neighbourhoods exist? How spread out are they?"
- **HINTS:**
  - `listings['neighbourhood_cleansed'].nunique()`.
  - Then `value_counts().head(20).plot.bar()` to see the top 20.
- **What you learn:** About 200 neighbourhoods. The top 20 hold half the listings. The tail is very long. This affects encoding choice.

Phase B — Encode and scale

Step 1 — Drop columns you do not need

- **WHAT:** Of 75 columns, keep maybe 20-25. Drop the rest now to make life easier.
- **HINTS:**
  - Keep: `id`, `host_id`, `host_since`, `neighbourhood_cleansed`, `neighbourhood_group_cleansed`, `latitude`, `longitude`, `room_type`, `accommodates`, `bedrooms`, `bathrooms`, `price`, `minimum_nights`, `number_of_reviews`, `reviews_per_month`, `last_review`, `first_review`, `review_scores_rating`, `description`, `availability_365`.
  - Drop the rest. Use `listings = listings[keep_cols]`.
- **WHY:** Most of the 75 columns are URLs, host pictures, summary text duplicates, calendar fields. Useless for price prediction.

Step 2 — Impute missing `bedrooms`

- **WHAT:** About 30% of `bedrooms` is missing. Decide how to fill.
- **TWO OPTIONS:**
  - **A — Fill with median (1.0)** for all missing. Simple.
  - **B — Fill with median PER `room_type`**. A "Private room" usually has 1 bedroom, an "Entire home" usually has 1 or 2. Smarter.
- **HINTS for option B:**
  - `listings.groupby('room_type')['bedrooms'].transform(lambda x: x.fillna(x.median()))`.
- **YOUR CHOICE:** Pick one. Write in your notebook WHY.

Step 3 — Impute missing `bathrooms` and `review_scores_rating`

- **WHAT:** Use median fill for the small leftover gaps.
- **HINTS:** `listings['bathrooms'] = listings['bathrooms'].fillna(listings['bathrooms'].median())`.
- **WARNING:** This is for inspection only. In Class 5, imputing goes INSIDE the Pipeline. Do NOT save the imputed version as your FINAL file before you check with the teacher.

Step 4 — One-hot encode `room_type`

- **WHAT:** Turn the 4 categories into 4 new 0/1 columns.
- **HINTS:**
  - Use `pd.get_dummies(listings, columns=['room_type'], prefix='room')`.
- **EXPECTED:** 4 new columns: `room_Entire home/apt`, `room_Private room`, `room_Shared room`, `room_Hotel room`.

Step 5 — One-hot encode `neighbourhood_group_cleansed`

- **WHAT:** Only 5 boroughs. Safe to one-hot.
- **HINTS:** `pd.get_dummies(..., columns=['neighbourhood_group_cleansed'], prefix='borough')`.
- **EXPECTED:** 5 new 0/1 columns.

Step 6 — Decide what to do with `neighbourhood_cleansed` (~200 values)

- **WHAT:** 200 categories is too many for one-hot.
- **TWO OPTIONS:**
  - **A — Keep only the top 30, group the rest as `"Other"`**. Then one-hot.
  - **B — Target encoding:** Replace each neighbourhood with the AVG `log_price` for that neighbourhood (compute from train only).
- **YOUR CHOICE:** Pick one. Write in your notebook WHY.
- **HINTS for option A:**
  - `top30 = listings['neighbourhood_cleansed'].value_counts().head(30).index`.
  - `listings['neighbourhood_cleansed'] = listings['neighbourhood_cleansed'].where(listings['ne 'Other')`.

Step 7 — Log-transform `price`

- **WHAT:** The target column for the model. Apply `np.log1p()`.
- **HINTS:**
  - `listings['log_price'] = np.log1p(listings['price'])`.
- **WHY `log1p` and not `log`?** `log(0)` is `−inf`. `log1p(x) = log(x + 1)` is safe for zeros. Even though we already dropped price = 0, this is good habit.

Step 8 — Scale numeric columns (preview only)

- **WHAT:** Use `StandardScaler` to make every numeric column have mean = 0, std = 1.
- **HINTS:**
  - `from sklearn.preprocessing import StandardScaler`.
  - `scaler = StandardScaler()`.
  - `scaler.fit_transform(listings[numeric_cols])`.
- **WARNING:** This is for inspection only. In Class 5, scaling goes INSIDE the Pipeline. Do NOT save the scaled version as your final file.

Step 9 — Save

- **WHAT:** Save to Drive. `listings.to_parquet('/content/drive/MyDrive/airbnb_lab/listings_step2.parquet`
- **Save the UNSCALED version.** Scaling will happen inside the Pipeline later.

Phase C — Make ONE chart for Marcus

Marcus's chart — "Price differs a lot by room type and borough"

A box plot of `price` (or `log_price`) split by `room_type`, with one panel per borough. Or just a grouped box plot.

- **HINTS:**
  - `sns.boxplot(data=listings, x='room_type', y='price', hue='neighbourhood_group_cleansed')`.
  - Or split into 5 small charts (one per borough).
  - Limit y-axis to a sensible range: `plt.ylim(0, 500)`.
- **Title:** "Price by room type and borough — Manhattan entire homes are 3x pricier than Bronx private rooms."
- **X-label:** Room type.
- **Y-label:** Price per night (USD).
- **Takeaway for Marcus:** "A 'fair price' depends on BOTH borough AND room type. A single city-wide hint will be wrong for most hosts."

Common mistakes in Class 2

Mistake	What goes wrong
Scale BEFORE train/test split	**Leakage.** Scaler sees test data.
One-hot encode `neighbourhood_cleansed` directly (200 cols)	Table explodes. Some columns are 1-hot for 5 rows only.
Forget `np.log1p` and use `np.log` on zeros	`np.log(0) = -inf`. Crash.
Save the scaled version	Class 5 will scale again inside the pipeline. Double scaling = wrong numbers.
Fill bedrooms with mean instead of median	The few 10-bedroom mansions pull the mean too high.

Self-check before Class 3

- ☐ One row per listing. About 38,000 rows.
- ☐ `bedrooms` and `bathrooms` no longer have missing values.
- ☐ `room_type` and `neighbourhood_group_cleansed` one-hot encoded.
- ☐ You decided what to do with `neighbourhood_cleansed`.
- ☐ `log_price` column exists.
- ☐ At least 3 exploratory charts.
- ☐ 1 chart for Marcus.
- ☐ `listings_step2.parquet` saved to Drive.

Class 3 — Feature Engineering

> **Scenario reminder:** Marcus says: "The columns in the raw data are not enough. The TRULY useful columns are not there. We must MAKE them. For example, the distance to Times Square — I bet listings closer to Times Square cost more. And the host experience — older hosts may charge more."

Your goal

Make NEW columns from the existing ones. These will help the model predict price.

Inputs

- `listings_step2.parquet`
- `reviews.csv.gz` (for review-based features)

Outputs

- `listings_step3.parquet` in Drive
- 3+ exploratory charts
- 1 explanatory chart for Marcus

Phase A — Explore the data first

Exploratory chart 1 — Map scatter (latitude vs longitude, colored by price)

- **Question:** "Where are the expensive listings on the NYC map?"
- **HINTS:**
  - `plt.scatter(listings['longitude'], listings['latitude'], c=listings['log_price'], s=2, cmap='viridis')`.

  - Add `plt.colorbar()`.
- **What you learn:** Manhattan is hotter. The shape of NYC appears like a real map.

Exploratory chart 2 — `log_price` by borough (after engineering)

- **Question:** "How does the average log_price compare across 5 boroughs?"
- **HINTS:**
  - `listings.groupby('neighbourhood_group_cleansed')['log_price'].mean().plot.bar()`.
- **What you learn:** Manhattan is highest. Bronx is lowest.

Exploratory chart 3 — Distance to Times Square histogram

- After you compute `distance_to_times_square_km` (Step 3 below), look at its distribution.
- **HINTS:** Histogram with `bins=50`.
- **What you learn:** Most listings are within 10 km. A few in Staten Island are 20-30 km away.

Exploratory chart 4 — Distance vs price (scatter)

- **Question:** "Are far listings cheaper?"
- **HINTS:**
  - Scatter `distance_to_times_square_km` on x-axis, `log_price` on y-axis.
  - Use `alpha=0.2` to see density.
- **What you learn:** Yes, price drops with distance. This is the most important location feature.

Phase B — Engineer the features

Step 1 — Create the target column `log_price`

- **WHAT:** You already made `log_price` in Class 2. Confirm it exists.
- **HINTS:**
  - `assert 'log_price' in listings.columns`.
  - `print(listings['log_price'].describe())`.
- **EXPECTED:** Mean around 4.8 (which is `log1p(120)` roughly). Min around 2.4, max around 7.3.

Step 2 — Host age in days

- **WHAT:** Make `host_age_days`. The number of days from `host_since` to the scrape date.
- **HINTS:**
  - Get the scrape date from `last_scraped` (or use today's date).
  - Subtract: `listings['last_scraped'] - listings['host_since']`.
  - Use `.dt.days` to convert from Timedelta to int.
- **WHY:** Experienced hosts charge differently than new hosts.

Step 3 — Compute Haversine distance to Times Square

This is the **biggest location feature.**

Times Square is the heart of tourist NYC. Its coordinates: **latitude 40.7580, longitude -73.9855**.

You need to write the Haversine formula. Skeleton:

```
import numpy as np
def haversine(lat1, lon1, lat2, lon2):
    R = 6371                    # Earth radius in km
    phi1 = np.radians(___)      # YOU fill in
    phi2 = np.radians(___)
```

```
    dphi = np.radians(___ - ___)
    dlam = np.radians(___ - ___)
    a = np.sin(dphi/2)**2 + np.cos(phi1)*np.cos(phi2) * np.sin(dlam/2)**2
    return 2 * R * np.arcsin(np.sqrt(a))
```

Then call it:

```
TIMES_SQ_LAT, TIMES_SQ_LON = 40.7580, -73.9855
listings['distance_to_times_square_km'] = haversine(
    listings['latitude'], listings['longitude'],
    ___, ___
)
```

- **WHY Haversine?** The Earth is a sphere. Straight-line distance on a flat map is wrong. Haversine gives the real distance.
- **EXPECTED:** `distance_to_times_square_km` between 0 (very close) and ~30 km (Staten Island).

Step 4 — Reviews per year

- **WHAT:** Make `reviews_per_year`. A measure of how active the listing is.
- **HINTS:**
  - You have `number_of_reviews` and `first_review`.
  - Years on platform = `(last_scraped - first_review).dt.days / 365`.
  - `listings['reviews_per_year'] = listings['number_of_reviews'] / years`.
  - If `years` is 0 or NaN, set the result to 0.
- **WHY:** A listing with 100 reviews in 1 year is busier than 100 reviews in 5 years. Busier listings might charge more.

Step 5 — Review text features

- **WHAT:** Join the `reviews` file. For each listing, compute two things:
  - `mean_review_length` — average number of characters in the comments.
  - `num_reviews_text` — count of reviews we actually have text for.
- **HINTS:**
  - Group reviews by `listing_id`: `reviews.groupby('listing_id')['comments'].agg(...)`.
  - For length: `lambda x: x.str.len().mean()`.
  - Then merge back into `listings` on `id == listing_id`.
- **WHY:** Listings with long, thoughtful reviews may be different from listings with short ones.
- **EXPECTED:** Most listings have a mean review length between 100 and 400 characters.

Step 6 — Description length

- **WHAT:** Make `description_length` from the `description` column.
- **HINTS:** `listings['description_length'] = listings['description'].fillna('').str.len()`.
- **WHY:** A long, detailed pitch may attract higher-paying guests. We will read the text properly in Module 7.

Step 7 — Domain ratio features (pick at least 2)

New column	What it is
`price_per_person`	`price / accommodates`
`bedrooms_per_person`	`bedrooms / accommodates`
`bath_per_bedroom`	`bathrooms / (bedrooms + 1)`
`is_superhost_neighbour`	For each neighbourhood, the % of superhosts

- **HINTS:**
  - Simple arithmetic for the first 3.

  - Add + 1 in the denominator to avoid divide-by-zero.
- **WHY:** A model can learn "more bathrooms per bedroom = luxury" only if YOU give it that ratio.

Step 8 — Save

- **HINTS:** `listings.to_parquet('/content/drive/MyDrive/airbnb_lab/listings_step3.parquet')`.

Phase C — Make ONE chart for Marcus

Marcus's chart — "Closer to Times Square, higher price"

A bar chart showing: bin distance into 5 buckets, show the mean `price` (in dollars, not log) in each bucket.

- **HINTS:**
  - `pd.cut(listings['distance_to_times_square_km'], bins=[0, 2, 5, 10, 15, 30])`.
  - GroupBy that, take the mean of `price`.
  - Bar chart.
- **Title:** "Median price by distance to Times Square — listings within 2 km charge 60% more than listings beyond 10 km."
- **X-label:** Distance to Times Square (km).
- **Y-label:** Median price per night (USD).
- **Takeaway for Marcus:** "The single biggest pricing signal is location. Our hint engine must factor in distance to the nearest tourist hub, not just the borough."

Common mistakes in Class 3

Mistake	What goes wrong
Use Euclidean distance instead of Haversine	Wrong distances. The Earth is round.
Mix up latitude and longitude in the Haversine call	Distances look crazy. NYC ends up "in the Pacific Ocean".
Divide by zero in ratios	Code crashes or makes `inf`. Always add + 1.
Use `reviews_per_month` as-is from the file	It is calculated per current activity, not historical. Make your own.
Use the actual price to engineer a feature, then use that feature to predict price	Leakage. The model gets near-100% but fails in real life.

Self-check before Class 4

- ☐ `host_age_days` exists. Range looks reasonable (0 to about 6000 days).
- ☐ `distance_to_times_square_km` looks reasonable (0 to 30 km).
- ☐ `reviews_per_year` exists. No NaN.
- ☐ `mean_review_length` joined from reviews file.
- ☐ At least 2 domain features.
- ☐ 3+ exploratory charts.
- ☐ 1 explanatory chart.
- ☐ `listings_step3.parquet` saved to Drive.

Class 4 — Feature Selection

> **Scenario reminder:** Now you have around 30 columns. Marcus says: “Too many. Some are duplicates. Some are useless. I want 10-15 GOOD columns. Pick them.”

Your goal

Pick the best 10-15 columns. Drop the rest. Justify every choice.

Inputs

- `listings_step3.parquet`

Outputs

- `listings_step4.parquet` in Drive (only the selected columns + `log_price`)
- 3+ exploratory charts (correlation heatmap, mutual info bars, importance bars)
- A short markdown report explaining your choices

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap of numeric columns

- **Question:** “Which columns say the same thing as each other?”
- **HINTS:**
  - Compute `listings[numeric_cols].corr()`.
  - Plot with `sns.heatmap()`.
  - Use `annot=True` and `fmt='.2f'`.
- **What you learn:** `accommodates`, `bedrooms`, `beds` are highly correlated. Pick one. `price_per_person` and `price` are also correlated — careful.

Exploratory chart 2 — Mutual information bar chart

- After you compute mutual info (Step 4 below), plot it.
- **HINTS:**
  - Sort the values, then bar chart.
- **What you learn:** Which columns predict `log_price` strongest.

Exploratory chart 3 — Random Forest feature importance

- After you train the RF (Step 5 below), plot the importances.
- **HINTS:** `pd.Series(rf.feature_importances_, index=...).sort_values().plot.barh()`.
- **What you learn:** A second opinion on feature importance.

Phase B — Select features

Step 1 — Split into train and test FIRST

- **WHAT:** Use `train_test_split`.
- **HINTS:**
  - `from sklearn.model_selection import train_test_split`.
  - `X = listings.drop(['price', 'log_price'], axis=1); y = listings['log_price']`.
  - Pass: `test_size=0.2`, `random_state=42`.

  - **NOTE:** No `stratify` for regression.
- **WHY:** All next steps use TRAIN ONLY. Otherwise leakage.

Step 2 — Drop ID and text columns (not features)

- **WHAT:** Columns like `id`, `host_id`, `description`, `last_scraped` are not numeric features.
- **HINTS:** Drop them from `X_train` and `X_test`. Keep `description` aside in another DataFrame — we will need it in Module 7.

Step 3 — Remove highly correlated columns

- **WHAT:** Drop one of each pair where |corr| > 0.9.
- **HINTS:**
  - Compute correlation matrix on numeric columns of `X_train`.
  - Get the upper triangle with `np.triu(...)`.
  - For each column, if any of its upper-triangle correlations is > 0.9, mark it for dropping.
- **EXAMPLE:** `accommodates` and `beds` are usually > 0.9. Keep `accommodates`, drop `beds`.

Step 4 — Rank by mutual information

- **WHAT:** Score each column by how much it tells you about `log_price`.
- **HINTS:**
  - This is regression, so use `mutual_info_regression`.
  - `from sklearn.feature_selection import mutual_info_regression`.
  - `mi = mutual_info_regression(X_train_numeric, y_train)`.
  - Put in a Series, sort.
- **EXPECTED:** `distance_to_times_square_km`, `room_Entire home/apt`, `accommodates`, `bedrooms`, `bathrooms`, `borough_Manhattan` should be near the top.

Step 5 — Random Forest importance (second opinion)

- **WHAT:** Train a small RF regressor. Look at `feature_importances_`.
- **HINTS:**
  - `from sklearn.ensemble import RandomForestRegressor`.
  - `RandomForestRegressor(n_estimators=50, max_depth=8, n_jobs=-1, random_state=42)`.
  - `.fit(X_train, y_train)`.
  - Look at `.feature_importances_`.

Step 6 — Pick the final 10-15 columns

- **WHAT:** Combine the rankings. Pick columns that:
  - Are high in mutual info, AND
  - Are high in RF importance, AND
  - Were not dropped by correlation pruning.
- **WRITE DOWN:** In your notebook, list the columns. Explain in 1 sentence why each one is in.

Step 7 — Save

- **HINTS:** Keep only the selected columns + `log_price` + `price` (raw, for analysis) + `description` (for Module 7). Save as `listings_step4.parquet` to Drive.

Phase C — Make ONE chart for Marcus

Marcus's chart — "These are the 10 most important columns for price"

A horizontal bar chart of your top 10 features and their RF importance score.

- **HINTS:**
  - Use the RF importance.
  - `sort_values()` then `.head(10)` then `.plot.barh()`.
- **Title:** "Top 10 predictors of nightly price."
- **Y-axis:** column names.
- **X-axis:** Random Forest importance.
- **Takeaway:** "Room type, size (accommodates and bedrooms), and distance to Times Square explain most of the price. Host history and review count add fine-tuning."

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on the FULL data	Leakage.
Use `mutual_info_classif` instead of `mutual_info_regression`	Wrong scores. Price is continuous.
Drop columns without writing why	Module 4 students will not understand.
Keep `price` AND `log_price` as features	Predicting `log_price` from `price`. 100% useless model.
Drop a high-mutual-info column	Big mistake. Check why it is high before dropping.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 10-15 columns remain + `log_price`.
- ☐ You wrote down WHY for each kept column.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Marcus.
- ☐ `listings_step4.parquet` saved to Drive.

Class 5 — Pipelines

> **Scenario reminder:** Marcus says: "Your cleaning code is in 4 different notebooks. When a new host posts a listing tomorrow morning, you cannot copy 4 notebooks to the server. We need ONE object that takes a raw listing and returns a price hint."

Your goal

Put EVERY cleaning step inside ONE Pipeline object. Production-ready code.

Inputs

- `listings_step4.parquet` (selected columns)

Outputs

- `airbnb_pipeline.joblib` saved in Drive
- 1 chart of predicted vs actual price
- A pipeline that takes RAW input and produces a price prediction

---

Phase A — Explore the data first

Exploratory chart 1 — Predicted vs actual log_price (after training)

- After Step 6, plot a scatter.
- **HINTS:**
  - `plt.scatter(y_test, y_pred, alpha=0.2)`.
  - Add a 45-degree line: `plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')`.
- **What you learn:** A good model has points near the red line.

Exploratory chart 2 — Residual histogram

- **WHAT:** A "residual" is `actual - predicted`. Plot the histogram.
- **HINTS:**
  - `residuals = y_test - y_pred`.
  - `plt.hist(residuals, bins=50)`.
- **What you learn:** The residuals should look like a bell curve around 0. If they are skewed, the model has systematic bias.

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric and which are categorical

- **HINTS:** Make two lists.
- **EXAMPLE:**
  - numeric: `accommodates`, `bedrooms`, `bathrooms`, `distance_to_times_square_km`, `host_age_days`, `reviews_per_year`, `mean_review_length`, `review_scores_rating`, `availability_365`.
  - categorical: `room_type`, `neighbourhood_group_cleansed`, possibly `neighbourhood_cleansed` (grouped).

Step 2 — Build the numeric mini-pipeline

- **WHAT:** 2 steps: imputer (fill missing) + scaler.
- **HINTS:**
  - `from sklearn.pipeline import Pipeline`.
  - `from sklearn.impute import SimpleImputer`.
  - `from sklearn.preprocessing import StandardScaler`.
  - Skeleton:

```
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='___')),
    ('scaler',  StandardScaler()),
])
```

  - Fill in the blank: best strategy for numeric? (`'median'`)

Step 3 — Build the categorical mini-pipeline

- **WHAT:** 2 steps: imputer + one-hot encoder.

- **HINTS:**
  - `from sklearn.preprocessing import OneHotEncoder`.
  - Skeleton:
    ```
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('onehot',  OneHotEncoder(handle_unknown='___', sparse_output=___)),
    ])
    ```
  - Fill in blanks. (Strategy `'most_frequent'`, handle_unknown `'ignore'`, sparse_output `False`.)
- **WHY `handle_unknown='ignore'`?** In production, a new neighbourhood name may appear. We do not want the code to crash.

Step 4 — Combine into a ColumnTransformer

- **HINTS:**
  - `from sklearn.compose import ColumnTransformer`.
  - It takes a list of tuples: `(name, transformer, list_of_columns)`.

Step 5 — Add the model on top

- **HINTS:**
  - For a baseline regression model, use `Ridge`.
  - `from sklearn.linear_model import Ridge`.
  - `Ridge(alpha=1.0, random_state=42)`.
  - Put it in a Pipeline with the preprocessor.
- **WHY Ridge and not plain LinearRegression?** Ridge handles correlated columns better. It is a simple, fast, safe baseline.

Step 6 — Train and evaluate

- **HINTS:**
  - `full_pipeline.fit(X_train, y_train)`.
  - `y_pred = full_pipeline.predict(X_test)`.
  - For regression, use these metrics:
    - `from sklearn.metrics import mean_absolute_error, r2_score`.
    - MAE on `log_price` should be around 0.30 to 0.45.
    - R^2 should be around 0.50 to 0.65.
- **EXPECTED:** Module 4 improves this with gradient boosting.

Step 7 — Convert log_price back to dollars for Marcus

- **WHAT:** Marcus speaks dollars, not logs. Show him predictions in real currency.
- **HINTS:**
  - `price_pred = np.expm1(y_pred)`.
  - `price_actual = np.expm1(y_test)`.
  - `mae_dollars = mean_absolute_error(price_actual, price_pred)`.
- **EXPECTED:** A MAE of about $40 to $60. Means our hint is on average $50 off.

Step 8 — Save the trained pipeline

- **HINTS:**
  - `import joblib`.
  - `joblib.dump(full_pipeline, '/content/drive/MyDrive/airbnb_lab/airbnb_pipeline.joblib')`.

Phase C — Make ONE chart for Marcus

Marcus's chart — "How close is our hint to the real price?"

A scatter plot of predicted vs actual price (in dollars, after `np.expm1`).

- **HINTS:**
  - X-axis: actual price. Y-axis: predicted price.
  - Add the 45-degree line in red.
  - Limit both axes to $0-$500 (the area where most listings are).
- **Title:** "Price hint accuracy — average error of $XX per night."
- **Takeaway for Marcus:** "On average, our hint is within $XX of the actual market price. Good enough for a first version. Module 4 will improve accuracy."

Common mistakes in Class 5

Mistake	What goes wrong
Build the pipeline AFTER you scaled the data manually	Pipeline scales it twice. Numbers wrong.
Forget `sparse_output=False` in OneHotEncoder	Some downstream code breaks.
Train on `price`, evaluate on `log_price` (or vice versa)	Metric numbers look impossible.
Forget `random_state`	Results change every run. Hard to debug.
Forget to invert the log with `np.expm1` before showing dollars	Numbers look like 4.7 (a log value) instead of $109. Confusing for Marcus.

Self-check before Class 6

- ☐ Pipeline has preprocessor + Ridge model.
- ☐ Preprocessor has numeric + categorical transformers.
- ☐ `handle_unknown='ignore'` set.
- ☐ You converted predictions back to dollars with `np.expm1`.
- ☐ Predicted-vs-actual scatter chart + residual histogram.
- ☐ `airbnb_pipeline.joblib` saved to Drive.

Class 6 — End-to-End Lab (THE BIG ONE)

> **Scenario reminder:** Marcus is in the meeting room. He wants the FINAL clean dataset on his desk in 90 minutes. This is the lab.

Your goal

Take the raw Inside Airbnb files. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Inputs

- The raw `listings.csv.gz` and `reviews.csv.gz` in your Drive folder

Outputs

- `airbnb_clean.parquet` (~38,000 rows x 21 columns) in Drive
- `findings.md` (~1 page)
- Your full Colab notebook

Phase A — Explore the data first (10 minutes)

You will reuse charts from Classes 1-5. Pick 3 of them. Put them all on ONE PAGE of your notebook with markdown headers: 1. NYC map scatter colored by `log_price` (the location story) 2. Distance to Times Square vs price (your most important chart) 3. Top 10 most important features

These 3 charts tell Marcus the whole story.

Phase B — Build the final dataset (70 minutes)

Required output schema

Your `airbnb_clean.parquet` MUST have these 21 columns, with these names and types:

#	Column	Type	Source
1	`listing_id`	int	listings (renamed from `id`)
2	`host_id`	int	listings
3	`neighbourhood`	string	listings (cleansed, top-30 + Other)
4	`borough`	string	listings (neighbourhood_group_cleansed)
5	`room_type`	string	listings
6	`latitude`	float	listings
7	`longitude`	float	listings
8	`accommodates`	int	listings
9	`bedrooms`	float	listings (imputed)
10	`bathrooms`	float	parsed from `bathrooms_text`
11	`minimum_nights`	int	listings
12	`availability_365`	int	listings
13	`number_of_reviews`	int	listings
14	`reviews_per_year`	float	engineered
15	`review_scores_rating`	float	listings (imputed)
16	`host_age_days`	int	engineered (from host_since)
17	`distance_to_times_square_km`	float	Haversine
18	`mean_review_length`	float	engineered from reviews file
19	`description`	string	listings (kept for Module 7)
20	`price`	float	cleaned (no $ no `,`)
21	`log_price`	float	TARGET (engineered, `np.log1p(price)`)

Lab phases (timed)

Phase	Time	What you do
1. Load	5 min	Load listings and reviews. Confirm row counts.
2. Clean price + bathrooms	15 min	Strip $ and `,`. Parse bathrooms_text. Drop bad rows.
3. Impute missing	10 min	Median fill for `bedrooms` (by room_type) and others.

Phase	Time	What you do
4. Group rare neighbourhoods	5 min	Top 30 + “Other”.
5. Geo + Haversine	15 min	Compute distance_to_times_square_km.
6. Date features	10 min	host_age_days, reviews_per_year.
7. Review features	10 min	mean_review_length from reviews file.
8. Save + findings	10 min	Validate schema. Save `.parquet`. Write findings.

Total: 90 minutes.

Phase C — Findings report for Marcus (10 minutes)

Write `findings.md` with these sections:

Final numbers

- Final row count: ____
- Median price: $____
- Median distance to Times Square: ____ km
- `log_price` mean: ____ (should be around 4.8)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart (the distance to Times Square vs price one).

Grading rubric (100 points)

Item	Points
All 21 columns exist with the right names and dtypes	25
Cleaning decisions written in markdown	10
Pipeline is reproducible (re-run from raw files in ONE command)	15
Haversine distance correct (within 0.5 km of the geopy library)	10
`price` outliers handled with a clear cutoff and explanation	10
At least 3 exploratory + 1 explanatory chart per class	**15**
`findings.md` has insights, not just numbers	10
Code is clean (comments, sensible variable names)	5

Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `airbnb_clean.parquet` - Your Colab notebook (`File > Download > Download .ipynb`) - `findings.md`

Tips for the whole module

1. **Do NOT copy-paste code from this guide.** This guide gives you HINTS only. Write the code yourself. You will learn much faster.
2. **Save your notebook to Drive often.** Colab disconnects after 12 hours.
3. **Save intermediate `.parquet` files to Drive** (listings_step1, listings_step2, etc.). If something breaks, you do not redo everything.
4. **Pair-program.** One student types, one reads. Switch every 20 minutes.
5. **Make a chart BEFORE you write cleaning code.** You must SEE the problem before you can fix it.
6. **Make a chart AFTER each step.** Confirm your code did what you expected.
7. **Always convert `log_price` back to dollars with `np.expm1` before showing Marcus.** He thinks in dollars, not logs.
8. **Ask the mentor early.** If you are stuck for 20 minutes, ask. Do not waste an afternoon.

Good luck.